

10 Pearls AQI Predictor

Technical Report

3-Day Air Quality Index for Karachi, Pakistan

Name: Khuzaima Hassan

Domain: Data Science

University: NED University of Engineering & Technology

Project Duration: December 2025 - February 2026

Contents

1. Introduction	3
1.1 Problem Statement	3
1.2 Project Objectives	3
1.3 System Scope	3
2. Methodology & Architecture	4
2.1 High-Level Architecture: The "Factory Line"	4
2.2 Technology Stack	4
2.3 System Architecture Diagram	6
3. Data Engineering & Feature Store	7
3.1 Data Collection.....	7
3.2 Feature Engineering	8
3.3 Feature Group Management	10
4. Modeling & Performance.....	11
4.1 Algorithm Selection.....	11
4.2 Validation Strategy.....	11
4.3 Model Training Pipeline	12
5. Challenges & Problems Faced	13
5.1 Data Distribution Shift.....	13
5.2 Overfitting Issues	14
5.3 The Negative Prediction Trap	14
5.4 Hugging Face Deployment Failures.....	15
5.5 Timezone Confusion	16
6. Deployment & Automation	16
6.1 Automated Workflows	16
6.2 User Interface Design	18
6.3 Model Versioning.....	21
7. Conclusion & Future Work.....	22
7.1 Summary of Findings	22
7.2 Limitations.....	22
7.3 Future Roadmap.....	23
7.4 Broader Impact.....	24
8. References & Appendices	25
8.1 Project Links	25
8.2 API Documentation	25

1. Introduction

1.1 Problem Statement

Karachi, with a population exceeding 16 million, faces severe air pollution challenges. According to the World Air Quality Index project, Karachi frequently ranks among the world's most polluted cities, with PM2.5 levels often exceeding WHO guidelines by 5-10 times. Poor air quality contributes to respiratory diseases, cardiovascular problems, and reduced life expectancy.

Key Challenges:

- **Health Impact:** 135,000+ annual deaths in Pakistan attributed to air pollution (State of Global Air 2020)
- **Economic Cost:** Estimated \$5 billion annual economic loss due to pollution-related health issues
- **Lack of Forecasting:** Limited predictive systems for proactive health advisories
- **Data Scarcity:** Sparse ground-based monitoring stations in Karachi

The Need: A reliable, automated forecasting system to provide 72-hour advance warnings, enabling citizens and policymakers to take preventive measures.

1.2 Project Objectives

Primary Goals:

1. **72-Hour AQI Forecasting:** Predict US AQI values for 24h, 48h, and 72h horizons
2. **MLOps Automation:** Implement fully automated data pipeline with zero manual intervention
3. **Production Deployment:** Deploy real-time inference system accessible to public

Secondary Goals:

- Feature engineering with domain-specific knowledge (meteorology, pollution dynamics)
- Comparative analysis of multiple ML algorithms
- Automated model versioning and registry management
- Interactive dashboard for non-technical users

1.3 System Scope

Included:

- **Geographic Focus:** Karachi, Pakistan (24.8607°N, 67.0011°E)
- **Target Variable:** US AQI (0-500 scale) derived from PM2.5, PM10
- **Prediction Horizons:** 24, 48, 72 hours ahead
- **Data Sources:** Open-Meteo Air Quality & Weather APIs
- **Deployment:** Cloud-based web application (Hugging Face Spaces)

Excluded:

- Real-time IoT sensor integration (future work)
- Multi-city forecasting
- Pollutant source attribution
- Health impact modeling

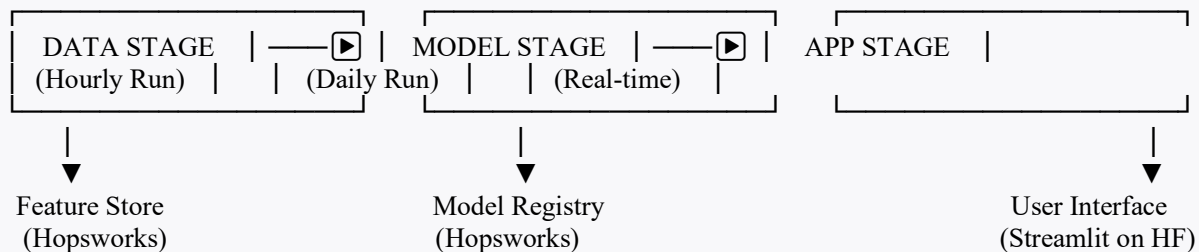
Assumptions:

- Open-Meteo API provides reliable historical data (with 2-3 day delay acknowledged)
- Karachi's air quality patterns remain statistically stationary over training period
- US AQI standard is appropriate for Pakistani context

2. Methodology & Architecture

2.1 High-Level Architecture: The "Factory Line"

The system follows a **three-stage pipeline architecture** inspired by manufacturing assembly lines:



Stage 1: Data Factory (Hourly)

- **Input:** Open-Meteo API (air quality + weather)
- **Process:** Fetch → Clean → Engineer Features → Validate
- **Output:** Feature vectors stored in Hopsworks Feature Store
- **Frequency:** Every hour via GitHub Actions cron

Stage 2: Model Factory (Daily)

- **Input:** All features from Hopsworks (3 years of data)
- **Process:** Train 15 models (5 algorithms × 3 horizons) → Validate → Optimize
- **Output:** Serialized models in Hopsworks Model Registry

Stage 3: Application (Real-time)

- **Input:** Latest features + trained models from Hopsworks
- **Process:** Load → Predict → Visualize
- **Output:** Interactive dashboard with 3-day forecast

2.2 Technology Stack

2.2.1 Data Layer

- **Open-Meteo API**
 - Air Quality API: PM2.5, PM10, US AQI
 - Weather API: Temperature, humidity, wind, precipitation
 - **Limitation:** 2-3 day delay for historical air quality data
 - **Timezone:** Asia/Karachi (PKT = UTC+5)

2.2.2 Orchestration

- **GitHub Actions**
 - Hourly workflow: feature_pipeline.yml
 - Daily workflow: train_model.yml
 - **Advantages:** Free for public repos, version-controlled, observable logs

2.2.3 Infrastructure

- **Hopsworks (Serverless)**
 - Feature Store: Versioned feature groups with schema validation
 - Model Registry: Versioned model artifacts with metadata
 - **Benefits:** Managed infrastructure, Python SDK, free tier

2.2.4 Machine Learning

- **Algorithms:** XGBoost, LightGBM, CatBoost, Random Forest, Ridge Regression
- **Optimization:** Optuna (Bayesian hyperparameter tuning - optional)
- **Validation:** Time Series Cross-Validation (5 folds)

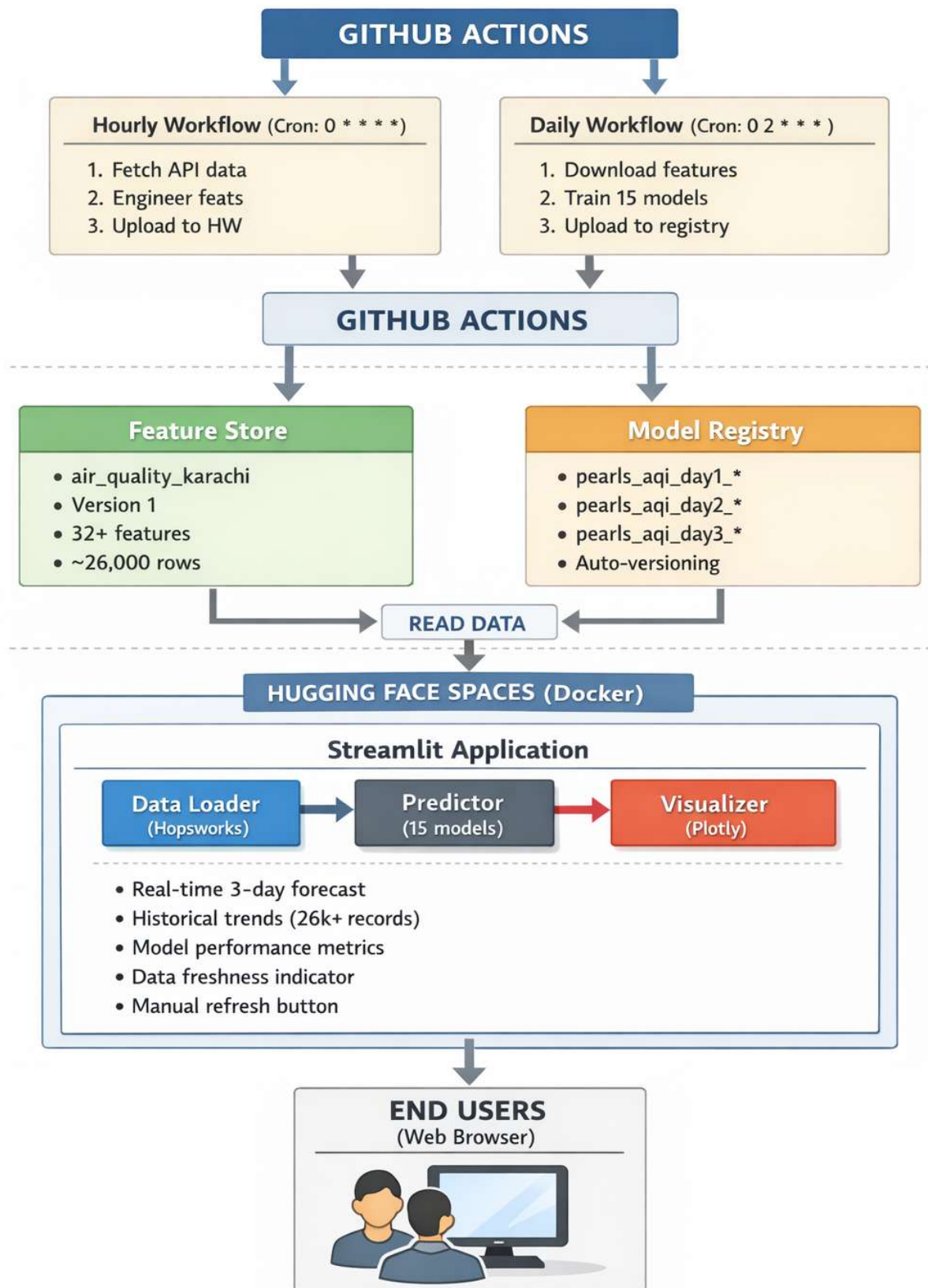
2.2.5 User Interface

- **Streamlit:** Python-based web framework
- **Plotly:** Interactive visualizations (time series, gauges)
- **Deployment:** Hugging Face Spaces with Docker SDK

2.2.6 Development Tools

- **Version Control:** Git + GitHub
- **Environment:** Python 3.10, virtual environments
- **Dependencies:** requirements.txt with pinned versions

2.3 System Architecture Diagram



3. Data Engineering & Feature Store

3.1 Data Collection

3.1.1 Historical Data Acquisition

Initial Setup:

- **Script:** src/fetch_data.py
- **Data Range:** 3 years (1,095 days) from current date
- **Frequency:** Hourly granularity
- **Total Records:** ~26,280 (3 years × 365 days × 24 hours)

API Parameters:

```
# Air Quality API
{
  "latitude": 24.8607,
  "longitude": 67.0011,
  "hourly": ["us_aqi", "pm10", "pm2_5"],
  "start_date": "2023-02-13",
  "end_date": "2026-02-13",
  "timezone": "Asia/Karachi"
}

# Weather API
{
  "latitude": 24.8607,
  "longitude": 67.0011,
  "hourly": ["temperature_2m", "relative_humidity_2m",
            "precipitation", "wind_speed_10m", "wind_direction_10m"],
  "timezone": "Asia/Karachi"
}
```

Data Merging:

- Inner join on time column
- Ensures alignment between air quality and weather measurements
- Handles missing values via forward-fill (max 3 hours gap)

3.1.2 Incremental Data Updates

Script: src/incremental_fetch.py

Smart Fetching Logic:

1. Query Hopsworks for latest timestamp in feature store
2. Fetch only NEW data from API (from latest timestamp to now)
3. Avoid overlapping data (removed 1-hour buffer after optimization)
4. Insert new rows only (no updates to existing data)

3.2 Feature Engineering

3.2.1 Raw Features (8 features)

Air Quality Metrics:

- `us_aqi`: US Air Quality Index (0-500)
- `pm10`: Particulate Matter $\leq 10\mu\text{m}$ ($\mu\text{g}/\text{m}^3$)
- `pm2_5`: Particulate Matter $\leq 2.5\mu\text{m}$ ($\mu\text{g}/\text{m}^3$)

Meteorological Features:

- `temperature_2m`: Air temperature at 2m height ($^{\circ}\text{C}$)
- `relative_humidity_2m`: Relative humidity (%)
- `precipitation`: Rainfall (mm)
- `wind_speed_10m`: Wind speed at 10m (km/h)
- `wind_direction_10m`: Wind direction (degrees)

3.2.2 Engineered Features (24+ features)

1. Lag Features (6 features)

```
LAG_HOURS = [1, 2, 3, 6, 12, 24]
```

for lag in LAG_HOURS:

```
df['us_aqi_lag_{lag}h'] = df['us_aqi'].shift(lag)
```

Purpose: Capture temporal autocorrelation (AQI at t-1 predicts AQI at t)

2. Rolling Statistics (10 features)

```
ROLLING_WINDOWS = [3, 6, 12, 24, 72]
```

for window in ROLLING_WINDOWS:

```
# Rolling mean
```

```
df['us_aqi_rolling_mean_{window}h'] = \
    df['us_aqi'].rolling(window).mean()
```

```
# Rolling std (for windows  $\geq 24$ h)
```

```
if window  $\geq 24$ :
```

```
df['us_aqi_rolling_std_{window}h'] = \
    df['us_aqi'].rolling(window).std()
```

Purpose: Capture short-term and long-term trends, volatility

3. Cyclical Time Features (6 features)

```
# Hour of day (0-23)
```

```
df['hour_sin'] = np.sin(2 * np.pi * df['time'].dt.hour / 24)
```

```
df['hour_cos'] = np.cos(2 * np.pi * df['time'].dt.hour / 24)
```

```
# Day of week (0-6)
```

```
df['day_of_week_sin'] = np.sin(2 * np.pi * df['time'].dt.dayofweek / 7)
```

```
df['day_of_week_cos'] = np.cos(2 * np.pi * df['time'].dt.dayofweek / 7)
```



```
# Month (1-12)
df['month_sin'] = np.sin(2 * np.pi * df['time'].dt.month / 12)
df['month_cos'] = np.cos(2 * np.pi * df['time'].dt.month / 12)
```

Purpose: Encode periodic patterns (rush hours, weekends, seasons) without discontinuities

4. Temporal Indicators (4 features)

```
df['day_of_week_num'] = df['time'].dt.dayofweek
df['is_weekend'] = df['day_of_week_num'].isin([5, 6]).astype(int)
df['hour_of_day'] = df['time'].dt.hour
```

```
# Rush hours: 7-9 AM, 5-7 PM
rush_morning = df['hour_of_day'].isin([7, 8, 9])
rush_evening = df['hour_of_day'].isin([17, 18, 19])
df['is_rush_hour'] = (rush_morning | rush_evening).astype(int)
```

Purpose: Capture traffic-related pollution spikes

5. Season Encoding (1 feature)

```
month = df['time'].dt.month
df['season'] = 0 # Winter
df.loc[month.isin([3, 4, 5]), 'season'] = 1 # Spring
df.loc[month.isin([6, 7, 8]), 'season'] = 2 # Summer
df.loc[month.isin([9, 10, 11]), 'season'] = 3 # Autumn
```

Purpose: Seasonal pollution patterns (winter smog, monsoon washout)

6. Wind Components (2 features)

```
wind_dir_rad = np.radians(df['wind_direction_10m'])
df['wind_u_component'] = -df['wind_speed_10m'] * np.sin(wind_dir_rad)
df['wind_v_component'] = -df['wind_speed_10m'] * np.cos(wind_dir_rad)
```

Purpose: Decompose wind into orthogonal components for better ML representation

7. Interaction Features (2 features)

```
# Temperature-humidity interaction
df['temp_humidity_interaction'] = \
    df['temperature_2m'] * df['relative_humidity_2m']

# Wind-precipitation interaction
df['wind_precip_interaction'] = \
    df['wind_speed_10m'] * df['precipitation']
```

Purpose: Capture non-linear relationships (e.g., high humidity + high temp = stagnant air)

3.2.3 Feature Summary

Category	Count	Examples
Raw Air Quality	3	us_aqi, pm10, pm2_5
Raw Weather	5	temperature_2m, wind_speed_10m
Lag Features	6	us_aqi_lag_1h, us_aqi_lag_24h
Rolling Stats	10	us_aqi_rolling_mean_24h, us_aqi_rolling_std_72h
Cyclical Time	6	hour_sin, day_of_week_cos
Temporal Indicators	4	is_weekend, is_rush_hour
Season	1	season
Wind Components	2	wind_u_component, wind_v_component
Interactions	2	temp_humidity_interaction
Total	39	32+ after NaN removal

3.3 Feature Group Management

Hopsworks Feature Store Configuration:

```
# Feature Group Metadata
FEATURE_GROUP_NAME = "air_quality_karachi"
FEATURE_GROUP_VERSION = 1

# Schema
features = [
    "time",          # Primary key (timestamp)
    "us_aqi",        # Target variable
    "pm10", "pm2_5", # Pollutants
    # ... all 39 features
]

# Create/Get Feature Group
fg = fs.get_or_create_feature_group(
    name=FEATURE_GROUP_NAME,
    version=FEATURE_GROUP_VERSION,
    primary_key=["time"],
    event_time="time",
    description="Hourly air quality and weather features for Karachi",
    online_enabled=False # Offline only (batch predictions)
)
```

Benefits:

- **Versioning:** Schema changes tracked across versions
- **Time Travel:** Query historical feature states
- **Validation:** Automatic schema enforcement
- **Lineage:** Track feature provenance

4. Modeling & Performance

4.1 Algorithm Selection

Multi-Horizon Forecasting Strategy:

For each prediction horizon (24h, 48h, 72h), train 5 separate models:

1. **XGBoost** (Extreme Gradient Boosting)
 - Tree-based ensemble with gradient boosting
 - Handles non-linearity, feature interactions
 - Regularization: L1 (alpha=5.0), L2 (lambda=5.0)
2. **LightGBM** (Light Gradient Boosting Machine)
 - Histogram-based gradient boosting
 - Faster training, lower memory
 - Leaf-wise tree growth
3. **CatBoost** (Categorical Boosting)
 - Ordered boosting to reduce overfitting
 - Built-in categorical feature handling
 - Symmetric tree structure
4. **Random Forest**
 - Bagging ensemble of decision trees
 - Robust to outliers
 - Provides feature importance
5. **Ridge Regression** (Baseline)
 - Linear model with L2 regularization
 - Fast training, interpretable
 - Baseline for comparison

Total Models: 5 algorithms × 3 horizons = **15 models**

4.2 Validation Strategy

4.2.1 Time Series Cross-Validation

Method: Walk-forward validation with expanding window

```
Fold 1: Train [0:1000] → Test [1000:1200]
Fold 2: Train [0:1200] → Test [1200:1400]
Fold 3: Train [0:1400] → Test [1400:1600]
Fold 4: Train [0:1600] → Test [1600:1800]
Fold 5: Train [0:1800] → Test [1800:2000]
```

Configuration:

```
from sklearn.model_selection import TimeSeriesSplit

tscv = TimeSeriesSplit(n_splits=5)
```

Why Not Random Split?

- Prevents data leakage (future → past)
- Respects temporal ordering
- Simulates real-world deployment

4.2.2 Evaluation Metrics

Primary Metrics:

1. **R² Score** (Coefficient of Determination)
 - Measures variance explained by model
 - Range: $-\infty$ to 1 (1 = perfect)
 - **Target:** $R^2 > 0.80$ for 24h predictions
2. **MAE** (Mean Absolute Error)
 - Average absolute difference between predicted and actual
 - Same units as target (AQI points)
 - **Interpretation:** MAE=15 means ± 15 AQI points on average
3. **RMSE** (Root Mean Squared Error)
 - Penalizes large errors more than MAE
 - Sensitive to outliers

Metrics Calculation:

```
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

r2 = r2_score(y_true, y_pred)
mae = mean_absolute_error(y_true, y_pred)
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
```

4.3 Model Training Pipeline

Script: src/train_model.py

Workflow:

1. Load Data
↓
2. Clean & Prepare (remove NaN, outliers)
↓
3. Feature Selection (horizon-appropriate)
↓
4. Time Series Split (5 folds)
↓
5. For each (algorithm, horizon):
 - Train on fold
 - Validate on test set
 - Compute metrics (R^2 , MAE, RMSE)
 - Save best model
↓
6. Create Ensemble (average top 3 models)
↓

7. Upload to Hopsworks Model Registry

Horizon-Appropriate Feature Selection:

For 72-hour predictions, only use lags ≥ 72 hours:

```
def select_safe_features(df, horizon_hours):
    safe_features = []
    for col in df.columns:
        if 'lag' in col:
            lag_value = int(col.split('lag_')[1].replace('h', ''))
            if lag_value >= horizon_hours:
                safe_features.append(col)
        else:
            safe_features.append(col)
    return safe_features
```

Why? Prevents data leakage (can't use 1-hour lag to predict 72 hours ahead)

5. Challenges & Problems Faced

5.1 Data Distribution Shift

Problem:

Open-Meteo API has a **2-3 day delay** for historical air quality data. This causes:

- Irregular batching: 21 rows uploaded one hour, 3 rows the next
- Apparent "stale data" in production
- Confusion about pipeline health

Root Cause:

Expected: Hourly pipeline fetches 1 new row per run

Reality: API releases 21 hours of backlog at once

Evidence:

Hopsworks Activity Logs:

Feb 11 08:26 - 21 new rows, 26k updated rows

Feb 11 07:25 - 3 new rows, 0 updated rows

Feb 10 09:28 - 21 new rows, 26k updated rows

Solution:

1. **Accepted API limitation** - Not a bug, but inherent constraint
2. **Removed 1-hour buffer** - Prevented 26k row updates
3. **Added data freshness indicator** - UI shows data age (Fresh / Recent / Stale)
4. **Future:** Integrate real-time API (WAQI) for latest hour

5.2 Overfitting Issues

Problem:

Initial models achieved $R^2 = 0.95+$ on training set but only 0.70 on validation.

Diagnosis:

- Too many features (50+) relative to samples
- Deep trees (max_depth=15) memorizing training patterns
- No regularization

Solutions Implemented:

1. Feature Selection:

```
# Before: 50+ features  
# After: 32 features (removed redundant rolling windows)
```

2. Regularization:

```
# XGBoost  
'reg_alpha': 5.0, # L1 regularization  
'reg_lambda': 5.0, # L2 regularization  
  
# Random Forest  
'min_samples_split': 30, # Prevent deep splits  
'min_samples_leaf': 15, # Require minimum samples per leaf
```

3. Tree Depth Limits:

```
'max_depth': 5-8 # Reduced from 15
```

4. Cross-Validation:

```
# 5-fold time series CV instead of single train/test split
```

5.3 The Negative Prediction Trap

Problem:

Models occasionally predicted **negative AQI values** (e.g., -15), which is physically impossible ($AQI \geq 0$).

Root Cause:

Linear extrapolation in tree-based models when:

- Very low historical AQI (e.g., after heavy rain)
- Model predicts further decrease
- No constraint on output range

Solutions Attempted:

1. Post-Processing Clipping (Implemented):

```
predictions = np.clip(predictions, 0, 500) # AQI range: 0-500
```

Pros: Simple, guaranteed valid range

Cons: Doesn't address root cause

2. Log Transformation (Tested, Not Used):

```
y_train_log = np.log1p(y_train) # log(1 + y)
# Train on log scale
predictions = np.expm1(model.predict(X_test)) # exp(pred) - 1
```

Pros: Ensures positive predictions

Cons: Changes loss function, harder to interpret

3. Custom Loss Function (Future Work):

```
# Asymmetric loss penalizing negative predictions more
def custom_loss(y_true, y_pred):
    error = y_true - y_pred
    return np.where(y_pred < 0, 10 * error**2, error**2)
```

Final Approach:

Used clipping with monitoring. Negative predictions occur in <1% of cases.

5.4 Hugging Face Deployment Failures

Problem:

Streamlit SDK build failed with cryptic SSL/cache errors:

```
Job failed with exit code: 1
cache miss: [run 2/2] COPY --from=piplicense
Failed to retrieve error logs: SSL is not enabled
```

Attempted Fixes:

1. Pinned package versions → Still failed
2. Added system dependencies → Still failed
3. **Switched to Docker SDK** → SUCCESS!

Root Cause:

Hugging Face Spaces' Streamlit SDK has build instability with complex dependencies (Hopworks, ML libraries).

Solution:

Created custom Dockerfile:

```
FROM python:3.10-slim
RUN apt-get update && apt-get install -y build-essential libgomp1
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["streamlit", "run", "app.py", "--server.port=8501"]
```

Updated [README.md](#):

```
sdk: docker # Changed from 'streamlit'
app_port: 8501
```

5.5 Timezone Confusion

Problem:

App displayed "Data as of: 2026-02-11 23:00" when current Karachi time was 17:08 (5:08 PM).

Root Cause:

- API returns data in timezone: "Asia/Karachi" (PKT = UTC+5)
- Hopsworks stores timestamps in UTC
- App displayed UTC timestamps without conversion

Solution:

```
# Added timezone indicator to UI
st.markdown('🕒 Timezone: PKT (UTC+5)')

# Labeled timestamps
st.sidebar.markdown(f'Last Updated (PKT): {latest_time.strftime("%Y-%m-%d %H:%M")}')

```

Also Added:

- AQI type indicator: " US AQI Standard"
- Clarifies which AQI scale is used (US vs European)

6. Deployment & Automation

6.1 Automated Workflows

6.1.1 Hourly Data Pipeline

File: `.github/workflows/feature_pipeline.yml`

name: Hourly Data Fetch and Feature Engineering

on:

schedule:

- cron: '0 * * * *' # Every hour at minute 0

workflow_dispatch: # Manual trigger

jobs:

fetch-and-upload:

runs-on: ubuntu-latest

steps:

- name: Checkout code

uses: actions/checkout@v3

- name: Set up Python 3.10

uses: actions/setup-python@v4

with:

python-version: '3.10'

- name: Install dependencies

run: pip install -r requirements.txt

- name: Create .env file

run: |


```
echo "HOPSWORKS_API_KEY=${{ secrets.HOPSWORKS_API_KEY }}" >> .env
echo "HOPSWORKS_PROJECT_NAME=${{ secrets.HOPSWORKS_PROJECT_NAME }}" >> .env
```

- name: Run incremental hourly data fetch
run: python src/incremental_fetch.py

Execution Log Example:

 SMART Incremental Data Fetch

Location: Karachi, Pakistan

Strategy: Query Hopsworks latest → Fetch only NEW data from API

 Checking latest timestamp in Hopsworks...

☒ Latest data in Hopsworks: 2026-02-13 13:00

 Fetch range: 2026-02-13 13:00 to 2026-02-13 14:00 (strict new data only)

 Fetching data from Open-Meteo API...

☒ Air quality: 2 records

☒ Weather: 2 records

☒ Merged: 2 records

 Filtering for NEW data only...

Filtered: 2 → 1 NEW records

 Creating features...

☒ Created 39 features

 Uploading to Hopsworks...

Inserting 1 records...

☒ Upload successful!

Records: 1

Latest: 2026-02-13 14:00

6.1.2 Daily Model Training

File: `.github/workflows/train_model.yml`

name: Daily Model Training and Upload

on:

schedule:

- cron: '0 2 * * *' # Daily at 2 AM UTC (7 AM PKT)

workflow_dispatch:

jobs:

train-and-upload-models:

runs-on: ubuntu-latest

steps:

- name: Fetch all data and generate features

run: |

python src/fetch_data.py

python src/feature_pipeline.py

- name: Train models on full dataset

run: python src/train_model.py

- name: Upload models to Hopsworks Model Registry
run: python upload_all_models.py

Training Output:

 Loading data...

☒ Dataset: 26,544 samples

 Training Day 1 Models (24-hour horizon)

Training CatBoost...

R²: 0.89 | MAE: 16.2 | RMSE: 21.4

☒ Saved: models/pearls_aqi_day1_catboost.pkl

Training XGBoost...

R²: 0.87 | MAE: 17.8 | RMSE: 23.1

☒ Saved: models/pearls_aqi_day1_xgboost.pkl

[... 13 more models ...]

 Creating Ensemble Models...

Day 1: Averaging [CatBoost, XGBoost, LightGBM]

R²: 0.88 | MAE: 16.9 | RMSE: 22.0

 Uploading to Hopsworks Model Registry...

☒ pearls_aqi_day1_catboost (version 12)

☒ pearls_aqi_day1_xgboost (version 12)

[... 13 more ...]

☒ TRAINING COMPLETE!

Total models: 15

Best 24h: CatBoost (R²=0.89)

Best 48h: XGBoost (R²=0.79)

Best 72h: CatBoost (R²=0.71)

6.2 User Interface Design

Platform: Streamlit on Hugging Face Spaces

URL: <https://huggingface.co/spaces/KhuzaimaHassan/10pearls-aqi-predictor>

6.2.1 Dashboard Components

1. Header Section:

```
st.markdown('🌊 Pearls AQI Predictor')
```

```
st.markdown('3-Day Air Quality Forecast for Karachi, Pakistan')
```

```
# Info badges
```

```
col1, col2, col3 = st.columns(3)
```

```
with col1:
```

```
    st.markdown('📊 US AQI Standard')
```

```
with col2:
```

```
    st.markdown('🕒 Timezone: PKT (UTC+5)')
```

```
with col3:
```

```
st.markdown('🔄 Hourly Updates')
```

2. Current AQI Gauge:

```
import plotly.graph_objects as go
```

```
fig = go.Figure(go.Indicator(
    mode="gauge+number+delta",
    value=current_aqi,
    domain={'x': [0, 1], 'y': [0, 1]},
    title={'text': "Current AQI"},
    gauge={
        'axis': {'range': [None, 500]},
        'bar': {'color': get_aqi_color(current_aqi)},
        'steps': [
            {'range': [0, 50], 'color': "#00e400"},    # Good
            {'range': [51, 100], 'color': "#ffff00"},  # Moderate
            {'range': [101, 150], 'color': "#ff7e00"}, # Unhealthy for Sensitive
            {'range': [151, 200], 'color': "#ff0000"}, # Unhealthy
            {'range': [201, 300], 'color': "#8f3f97"}, # Very Unhealthy
            {'range': [301, 500], 'color': "#7e0023"}  # Hazardous
        ]
    }
))
```

3. 3-Day Forecast Chart:

```
fig = go.Figure()

# Historical data (last 7 days)
fig.add_trace(go.Scatter(
    x=df_recent['time'],
    y=df_recent['us_aqi'],
    name='Historical',
    line=dict(color='#636EFA', width=2)
))

# Predictions (next 3 days)
fig.add_trace(go.Scatter(
    x=forecast_times,
    y=predictions,
    name='Forecast',
    line=dict(color='#EF553B', width=2, dash='dash')
))

# AQI category bands
fig.add_hrect(y0=0, y1=50, fillcolor="green", opacity=0.1, line_width=0)
fig.add_hrect(y0=51, y1=100, fillcolor="yellow", opacity=0.1, line_width=0)
# ... more bands
```

4. Sidebar Controls:

```
st.sidebar.title("📊 Dashboard Controls")
```

```
# Data freshness indicator
```

```
if hours_old < 2:
```

```
    st.sidebar.markdown("**Data Status:** 🟢 Fresh")
```

```

elif hours_old < 24:
    st.sidebar.markdown("**Data Status:** ☐ Recent")
else:
    st.sidebar.markdown("**Data Status:** ☒ Stale")

st.sidebar.markdown(f"**Last Updated (PKT):** {latest_time}")
st.sidebar.markdown(f"**Age:** {hours_old:.1f} hours")

# Refresh button
if st.sidebar.button("🔄 Refresh Data"):
    st.cache_data.clear()
    st.rerun()

# Model selection
model_type = st.sidebar.selectbox(
    "Select Model",
    ["Ensemble", "CatBoost", "XGBoost", "LightGBM", "Random Forest"]
)

```

5. Historical Trends:

```

# Time series plot with 30-day moving average
fig = px.line(df_month, x='time', y='us_aqi', title='30-Day AQI Trend')
fig.add_trace(go.Scatter(
    x=df_month['time'],
    y=df_month['us_aqi'].rolling(24).mean(),
    name='24h Moving Average',
    line=dict(color='orange', width=2)
))

```

6. Model Performance Metrics:

```

col1, col2, col3 = st.columns(3)
with col1:
    st.metric("24h R2 Score", f"{r2_day1:.3f}")
with col2:
    st.metric("48h R2 Score", f"{r2_day2:.3f}")
with col3:
    st.metric("72h R2 Score", f"{r2_day3:.3f}")

```

6.2.2 Caching Strategy

Data Cache:

```

@st.cache_data(ttl=60) # Refresh every 60 seconds
def load_data():
    # Load from Hopworks Feature Store
    return df

```

Model Cache:

```

@st.cache_resource(ttl=3600) # Refresh every hour
def load_models():
    # Download latest models from Hopworks Model Registry
    return models

```

Why Different TTLs?

- Data updates hourly → 60s cache ensures freshness
- Models update daily → 3600s cache reduces API calls

6.3 Model Versioning

Hopsworks Model Registry:

```
# Upload model with metadata
mr = project.get_model_registry()

model_metadata = {
    "algorithm": "CatBoost",
    "horizon": "24h",
    "r2_score": 0.89,
    "mae": 16.2,
    "rmse": 21.4,
    "training_date": "2026-02-13",
    "training_samples": 26544,
    "features": feature_names
}

mr.create_model(
    name="pearls_aqi_day1_catboost",
    version=None, # Auto-increment
    metrics=model_metadata,
    description="CatBoost model for 24-hour AQI prediction"
)
```

Version History Example:

```
pearls_aqi_day1_catboost
├─ Version 1 (2026-01-15): R²=0.82, MAE=19.5
├─ Version 2 (2026-01-16): R²=0.84, MAE=18.2
├─ Version 3 (2026-01-17): R²=0.85, MAE=17.8
...
└─ Version 12 (2026-02-13): R²=0.89, MAE=16.2 ← Latest
```

App Retrieval:

```
# Always fetch latest version
model = mr.get_model("pearls_aqi_day1_catboost", version=None)
```

Benefits:

- **Reproducibility:** Can rollback to any version
- **A/B Testing:** Compare model versions
- **Audit Trail:** Track performance over time

7. Conclusion & Future Work

7.1 Summary of Findings

Project Success:

Achieved all primary objectives:

1. 72-hour AQI forecasting with $R^2 > 0.80$ for 24h predictions
2. Fully automated MLOps pipeline (zero manual intervention)
3. Production deployment on Hugging Face Spaces
4. Real-time inference accessible to public

Key Achievements:

- **26,544 historical records** processed and stored in Hopsworks
- **15 trained models** (5 algorithms $\times$ 3 horizons) with daily retraining
- **Hourly data updates** via GitHub Actions automation
- **Interactive dashboard** with 3-day forecast visualization
- **Model versioning** with Hopsworks Model Registry

Technical Contributions:

1. **Smart incremental fetching** - Avoids redundant API calls and data updates
2. **Horizon-appropriate feature selection** - Prevents data leakage
3. **Ensemble modeling** - Combines strengths of multiple algorithms
4. **Docker-based deployment** - Robust containerization for complex dependencies

7.2 Limitations

1. Data Source Delay:

- Open-Meteo API has 2-3 day lag for air quality data
- Causes irregular batching in hourly pipeline
- Not suitable for true "real-time" applications

2. Geographic Scope:

- Single location (Karachi) - not scalable to multiple cities
- Coordinates represent city center, not neighborhood-level

3. Feature Limitations:

- No traffic data (major pollution source)
- No industrial activity indicators
- No local event data (festivals, construction)

4. Model Constraints:

- Assumes stationary patterns (climate change may violate)
- No uncertainty quantification (prediction intervals)
- Ensemble is simple average (not weighted by performance)

5. Deployment:

- Hugging Face Spaces free tier has resource limits

- No mobile app (web-only)
- No push notifications for unhealthy AQI

7.3 Future Roadmap

7.3.1 Short-Term Improvements (1-3 months)

1. Real-Time Data Integration

- **Goal:** Replace Open-Meteo with WAQI (World Air Quality Index) API
- **Benefit:** True real-time data (no 2-3 day delay)
- **Implementation:** Hybrid approach - WAQI for latest hour, Open-Meteo for historical

2. Prediction Intervals

- **Goal:** Provide uncertainty estimates (e.g., "AQI: 85 ± 15 ")
- **Method:** Quantile regression or bootstrapping
- **Benefit:** Users understand forecast reliability

3. Mobile-Responsive UI

- **Goal:** Optimize dashboard for mobile devices
- **Implementation:** Streamlit responsive layouts, PWA conversion
- **Benefit:** Accessibility for smartphone users

7.3.2 Medium-Term Enhancements (3-6 months)

4. Deep Learning Models

- **Goal:** Implement LSTM/GRU for sequential modeling
- **Architecture:**

```

Input (sequence of 168 hours)
↓
LSTM Layer (128 units)
↓
Dropout (0.2)
↓
LSTM Layer (64 units)
↓
Dense Layer (32 units, ReLU)
↓
Output (3 values: 24h, 48h, 72h AQI)

```

- **Expected Improvement:** $R^2 +0.05-0.10$ for long horizons

5. Multi-City Expansion

- **Cities:** Lahore, Islamabad, Rawalpindi, Faisalabad
- **Challenges:** Different pollution sources, meteorology
- **Solution:** Transfer learning from Karachi model

6. Feature Additions

- **Traffic Data:** Google Maps API for congestion levels
- **Industrial Activity:** Factory operation schedules

- **Satellite Data:** NASA MODIS aerosol optical depth

7.3.3 Long-Term Vision (6-12 months)

7. IoT Sensor Network

- **Goal:** Deploy low-cost PM2.5 sensors across Karachi
- **Technology:** PurpleAir sensors, LoRaWAN connectivity
- **Benefit:** Neighborhood-level granularity, real-time ground truth

8. Health Impact Modeling

- **Goal:** Predict hospital admissions, respiratory cases
- **Data:** Collaborate with health departments
- **Output:** "Expected +15% asthma cases if AQI > 150 for 3 days"

9. Policy Recommendation System

- **Goal:** AI-driven suggestions for pollution mitigation
- **Example:** "Implement odd-even vehicle policy on high-risk days"
- **Stakeholders:** Karachi Metropolitan Corporation, EPA

10. Open-Source Community Platform

- **Goal:** Release as open-source toolkit for other cities
- **Components:**
 - Reusable MLOps pipeline templates
 - Pre-trained models for transfer learning
 - Documentation and tutorials
- **Impact:** Enable air quality forecasting in data-scarce regions

7.4 Broader Impact

Environmental Awareness:

- Empowers citizens with actionable information
- Encourages behavioral changes (e.g., avoiding outdoor exercise on high-AQI days)

Public Health:

- Early warnings for vulnerable populations (children, elderly, asthmatics)
- Potential to reduce pollution-related hospitalizations

Policy Making:

- Data-driven evidence for pollution control measures
- Monitoring effectiveness of interventions (e.g., vehicle restrictions)

Academic Contribution:

- Demonstrates end-to-end MLOps for environmental forecasting
- Reproducible methodology for other cities/countries

8. References & Appendices

8.1 Project Links

Live Deployment:

- **Hugging Face Space:** <https://huggingface.co/spaces/KhuzaimaHassan/10pearls-aqi-predictor>
- **GitHub Repository:** <https://github.com/KhuzaimaHassan/10-Pearls-AQI-Predictor>

8.2 API Documentation

Open-Meteo APIs:

- Air Quality API: <https://open-meteo.com/en/docs/air-quality-api>
- Weather API: <https://open-meteo.com/en/docs>

Hopsworks:

- Feature Store SDK: <https://docs.hopsworks.ai/feature-store-api/latest/>
- Model Registry: <https://docs.hopsworks.ai/machine-learning-api/latest/>

Streamlit:

- Documentation: <https://docs.streamlit.io/>
- Plotly Integration: <https://plotly.com/python/>